# Laplacian Solver

Generated by Doxygen 1.14.0

# Chapter 1

# Numerical Solver for -a u_xx - b u_yy = f

## 1.1 Overview

This project implements a numerical solver for the elliptic partial differential equation

$$-a\frac{\partial^2 u}{\partial x^2} - b\frac{\partial^2 u}{\partial y^2} = f(x, y)$$

on the unit square $[0, 1] \times [0, 1]$ with Dirichlet boundary conditions.

The solution is obtained using the **Seidel method**, and the eigenvalues of the discrete Laplacian operator are computed using the **power method** and **shifted power method**.

## 1.2 Main Modules

- seidel_solver.h : Implementation of the Seidel iterative solver.
- power_method.h : Power method for the largest eigenvalue.
- shifted_power_method.h : Shifted power method for the smallest eigenvalue.
- laplacian_operator.h : Discrete Laplacian operator.
- functions.h : Exact solution, source term, and boundary conditions.
- eigen_theory.h : Theoretical eigenvalues for verification.
- output_handler.h : Output routines (saving to binary file, plotting).
- solver_controller.h : High-level controller that runs the complete simulation.

## 1.3 Usage

The program is executed via `main.cpp`, which sets up the parameters:

- Coefficients: $a = 0.9$, $b = 1.1$
- Grid step: $h = 0.1$
- Tolerances: $10^{-6}$ for both eigenvalue and Seidel iterations
- Maximum iterations: 100000

The solver:

1. Solves the PDE using the Seidel method.
2. Saves the solution to `solution.bin` and generates plots via `plot_full.py`.
3. Computes the largest eigenvalue (using alternating-sign initial vector) and the smallest eigenvalue (using unit constant initial vector).

## 1.4 List of Files

- context.h
- eigen_theory.h
- functions.h
- initial_vector_type.h
- laplacian_operator.h
- main.cpp
- math_utils.h
- mesh.h
- output_handler.h
- plot_full.py
- power_method.h
- seidel_solver.h
- shifted_power_method.h
- solver_controller.h
- clock_wrapper.h

## 1.5 Notes

- The grid is uniform with $N = 1/h + 1$ nodes per side.
- Theoretical eigenvalues are given by:

$$\lambda_{\min} = \frac{4(a+b)}{h^2} \sin^2 \frac{\pi h}{2}, \qquad \lambda_{\max} = \frac{4(a+b)}{h^2} \cos^2 \frac{\pi h}{2}.$$

- The Python script `plot_full.py` requires `numpy` and `matplotlib`.

# Chapter 2

# Namespace Index

## 2.1 Namespace List

Here is a list of all namespaces with brief descriptions:


eigen_theory
Theoretical formulas for eigenvalues of the Laplace operator . . . . . ??
exact
Exact solution and right-hand side for the test equation . . . . . ??
math_utils
Auxiliary mathematical functions: dot product, norm, constants . . . . . ??
plot_full . . . . . ??

# Chapter 3

# Class Index

## 3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:


Clock
: Simple RAII timer for measuring elapsed time . . . . . ??

Context
: Contains all solver configuration parameters . . . . . ??

LaplacianOperator
: Discrete operator: -a u_xx - b u_yy . . . . . ??

OutputHandler
: Static class for outputting results: printing, saving to binary file, plotting . . . . . ??

PowerMethod::Params< Operator >
: Parameters for the power method . . . . . ??

PowerMethod
: Implements the power method for computing the largest eigenvalue . . . . . ??

SeidelSolver
: Solves the equation -a u_xx - b u_yy = f using the Seidel method . . . . . ??

ShiftedPowerMethod
: Computes the smallest eigenvalue using the shifted power method . . . . . ??

SolverController
: Controller managing the solution of the equation, eigenvalue computation, and output/saving of results . . . . . ??

UniformMesh
: Represents a uniform rectangular grid on [0,1]x[0,1] with a given step . . . . . ??

# Chapter 4

# File Index

## 4.1 File List

Here is a list of all files with brief descriptions:


clock_wrapper.h . . . . . ??
context.h . . . . . ??
eigen_theory.h . . . . . ??
functions.h . . . . . ??
initial_vector_type.h . . . . . ??
laplacian_operator.h . . . . . ??
main.cpp . . . . . ??
math_utils.h . . . . . ??
mesh.h . . . . . ??
output_handler.h . . . . . ??
plot_full.py
( ) . . . . . ??
power_method.h . . . . . ??
seidel_solver.h . . . . . ??
shifted_power_method.h . . . . . ??
solver_controller.h . . . . . ??

# Chapter 5

# Namespace Documentation

## 5.1 eigen_theory Namespace Reference

Theoretical formulas for eigenvalues of the Laplace operator.

### Functions

- DataType lambda_min (DataType coeff_x, DataType coeff_y, DataType h)

  *Computes the theoretical minimum eigenvalue.*
- DataType lambda_max (DataType coeff_x, DataType coeff_y, DataType h)

  *Computes the theoretical maximum eigenvalue.*
- std::tuple< DataType, DataType > lambda_max_min (DataType coeff_x, DataType coeff_y, DataType h)

  *Returns both eigenvalues as a tuple.*

### 5.1.1 Detailed Description

Theoretical formulas for eigenvalues of the Laplace operator.

### 5.1.2 Function Documentation

#### 5.1.2.1 lambda_max()

```
DataType eigen_theory::lambda_max (
            DataType coeff_x,
            DataType coeff_y,
            DataType h)  [inline]
```

Computes the theoretical maximum eigenvalue.

**Parameters**

| | | |
|---|---|---|
| | *coeff_x* | Coefficient a. |
| | *coeff_y* | Coefficient b. |
| | *h* | Grid step. |

**Returns**

Maximum eigenvalue: 4(a+b)/h^2 ∗ cos^2(pi∗h/2).

Definition at line 33 of file eigen_theory.h.

#### 5.1.2.2 lambda_max_min()

```
std::tuple< DataType, DataType > eigen_theory::lambda_max_min (
            DataType coeff_x,
            DataType coeff_y,
            DataType h)  [inline]
```

Returns both eigenvalues as a tuple.

**Parameters**

| | | |
|---|---|---|
| | *coeff_x* | Coefficient a. |
| | *coeff_y* | Coefficient b. |
| | *h* | Grid step. |

**Returns**

Tuple (lambda_max, lambda_min).

Definition at line 47 of file eigen_theory.h.

#### 5.1.2.3 lambda_min()

```
DataType eigen_theory::lambda_min (
            DataType coeff_x,
            DataType coeff_y,
            DataType h)  [inline]
```

Computes the theoretical minimum eigenvalue.

**Parameters**

| | | |
|---|---|---|
| | *coeff_x* | Coefficient a. |
| | *coeff_y* | Coefficient b. |
| | *h* | Grid step. |

**Returns**

Minimum eigenvalue: 4(a+b)/h^2 $*$ sin^2(pi$*$h/2).

Definition at line 19 of file eigen_theory.h.

## 5.2 exact Namespace Reference

Exact solution and right-hand side for the test equation.

**Functions**

- DataType u (DataType x, DataType y)

  *Exact solution u(x,y) = x^2 + cos^2(y∗x).*
- DataType source (DataType x, DataType y, DataType coeff_x, DataType coeff_y)

  *Right-hand side corresponding to the exact solution.*
- DataType boundary (DataType x, DataType y)

  *Boundary condition (same as exact solution).*

## 5.2.1 Detailed Description

Exact solution and right-hand side for the test equation.

## 5.2.2 Function Documentation

### 5.2.2.1 boundary()

```
DataType exact::boundary (
            DataType x,
            DataType y)  [inline]
```

Boundary condition (same as exact solution).

**Parameters**

| | | |
|---|---|---|
| | *x* | x-coordinate. |
| | *y* | y-coordinate. |

**Returns**

u(x,y).

Definition at line 40 of file functions.h.

### 5.2.2.2 source()

```
DataType exact::source (
            DataType x,
            DataType y,
            DataType coeff_x,
            DataType coeff_y)  [inline]
```

Right-hand side corresponding to the exact solution.

**Parameters**

| | | |
|---|---|---|
| | *x* | x-coordinate. |
| | *y* | y-coordinate. |
| | *coeff_x* | Coefficient a. |
| | *coeff_y* | Coefficient b. |

**Returns**

f(x,y) = -2a + 2 cos(2yx)(a y^2 + b x^2).

Definition at line 28 of file functions.h.

#### 5.2.2.3 u()

```
DataType exact::u (
            DataType x,
            DataType y)  [inline]
```

Exact solution u(x,y) = x^2 + cos^2(y∗x).

**Parameters**

| | | |
|---|---|---|
| | *x* | x-coordinate. |
| | *y* | y-coordinate. |

**Returns**

u(x,y).

Definition at line 16 of file functions.h.

## 5.3 math_utils Namespace Reference

Auxiliary mathematical functions: dot product, norm, constants.

### Functions

- DataType dot (const std::vector< DataType > &a, const std::vector< DataType > &b)

  *Dot product of two vectors.*
- DataType norm (const std::vector< DataType > &v)

  *Euclidean norm of a vector.*

### Variables

- constexpr long double PI = 3.14159265358979323846

  *Pi constant.*

### 5.3.1 Detailed Description

Auxiliary mathematical functions: dot product, norm, constants.

### 5.3.2 Function Documentation

#### 5.3.2.1 dot()

```
DataType math_utils::dot (
            const std::vector< DataType > & a,
            const std::vector< DataType > & b)  [inline]
```

Dot product of two vectors.

**Parameters**

| | | |
|---|---|---|
| | *a* | First vector. |
| | *b* | Second vector. |

**Returns**

Sum of a[i]∗b[i] over all elements.

**Note**

Uses OpenMP parallel reduction.

Definition at line 24 of file math_utils.h.

#### 5.3.2.2 norm()

```
DataType math_utils::norm (
            const std::vector< DataType > & v)  [inline]
```

Euclidean norm of a vector.

**Parameters**

| | | |
|---|---|---|
| | *v* | Input vector. |

**Returns**

sqrt(dot(v,v)).

Definition at line 38 of file math_utils.h.

### 5.3.3 Variable Documentation

#### 5.3.3.1 PI

```
long double math_utils::PI = 3.14159265358979323846  [constexpr]
```

Pi constant.

Definition at line 15 of file math_utils.h.

## 5.4 plot_full Namespace Reference

**Functions**

- np.ndarray u_exact (Union[float, np.ndarray] x, Union[float, np.ndarray] y)

  *Exact analytical solution of the problem.*
- Tuple[np.ndarray, np.ndarray, np.ndarray] read_full_solution (str filename)

  *Reads a binary solution file created by OutputHandler.save_full_solution.*
- None main ()

  *Main function of the script.*

### 5.4.1 Function Documentation

#### 5.4.1.1 main()

```
None plot_full.main ()
```

Main function of the script.

Performs reading of the file, computation of exact solution and error, creates and saves plots (comparison.png, error_map.png), and displays them on the screen.

Definition at line 60 of file plot_full.py.

#### 5.4.1.2 read_full_solution()

```
Tuple[np.ndarray, np.ndarray, np.ndarray] plot_full.read_full_solution (
            str filename)
```

Reads a binary solution file created by OutputHandler.save_full_solution.

**Parameters**

| | | |
|---|---|---|
| | *filename* | File name (e.g., 'solution.bin'). |

**Returns**

Tuple (X, Y, u_num), where X, Y are coordinate matrices (meshgrid), u_num is a 2D array of the numerical solution.

**Exceptions**

| | | |
|---|---|---|
| | *ValueError* | if the file is too short or has an invalid format. |

File format:

- first 8 bytes: number of nodes along x (nx, size_t)
- next 8 bytes: number of nodes along y (ny, size_t)
- then nx∗ny double values in row-major order (j=0..ny-1, i=0..nx-1)

Definition at line 27 of file plot_full.py.

#### 5.4.1.3 u_exact()

```
np.ndarray plot_full.u_exact (
            Union[float, np.ndarray] x,
            Union[float, np.ndarray] y)
```

Exact analytical solution of the problem.

**Parameters**

| | | |
|---|---|---|
| | *x* | x-coordinate (scalar or numpy array). |
| | *y* | y-coordinate (scalar or numpy array). |

**Returns**

u(x,y) = x^2 + cos^2(y∗x).

Definition at line 18 of file plot_full.py.

# Chapter 6

# Class Documentation

## 6.1 Clock Class Reference

Simple RAII timer for measuring elapsed time.

```
#include <clock_wrapper.h>
```

#### Public Types

- using TimePoint = std::chrono::steady_clock::time_point
- using ChronoClock = std::chrono::high_resolution_clock
- using Elapsed = std::chrono::duration<DataType>

#### Public Member Functions

- Clock ()

  *Constructor starts the timer.*
- Elapsed elapsed ()

  *Returns the time elapsed since construction or the last restart.*
- Elapsed restart ()

  *Restarts the timer and returns the elapsed time.*

### 6.1.1 Detailed Description

Simple RAII timer for measuring elapsed time.

Uses std::chrono::high_resolution_clock.

Definition at line 10 of file clock_wrapper.h.

### 6.1.2 Member Typedef Documentation

#### 6.1.2.1 ChronoClock

```
using Clock::ChronoClock = std::chrono::high_resolution_clock
```

Definition at line 13 of file clock_wrapper.h.

#### 6.1.2.2 Elapsed

```
using Clock::Elapsed = std::chrono::duration<DataType>
```

Definition at line 14 of file clock_wrapper.h.

#### 6.1.2.3 TimePoint

```
using Clock::TimePoint = std::chrono::steady_clock::time_point
```

Definition at line 12 of file clock_wrapper.h.

### 6.1.3 Constructor & Destructor Documentation

#### 6.1.3.1 Clock()

```
Clock::Clock () [inline]
```

Constructor starts the timer.

Definition at line 19 of file clock_wrapper.h.

### 6.1.4 Member Function Documentation

#### 6.1.4.1 elapsed()

```
Elapsed Clock::elapsed () [inline]
```

Returns the time elapsed since construction or the last restart.

**Returns**

Duration in seconds (DataType).

Definition at line 25 of file clock_wrapper.h.

#### 6.1.4.2 restart()

```
Elapsed Clock::restart () [inline]
```

Restarts the timer and returns the elapsed time.

**Returns**

Duration in seconds (DataType).

Definition at line 34 of file clock_wrapper.h.

The documentation for this class was generated from the following file:

- clock_wrapper.h

## 6.2 Context Struct Reference

Contains all solver configuration parameters.

```
#include <context.h>
```

**Public Attributes**

- UniformMesh mesh

  *The grid.*
- const DataType a

  *Coefficient for -u_xx.*
- const DataType b

  *Coefficient for -u_yy.*
- const DataType h

  *Grid step (duplicated from mesh).*
- const DataType tol_power

  *Tolerance for the power method.*
- const DataType tol_seidel

  *Tolerance for the Seidel method.*
- const size_t max_iter_power

  *Maximum number of iterations for the power method.*
- const size_t max_iter_seidel

  *Maximum number of iterations for the Seidel method.*

### 6.2.1 Detailed Description

Contains all solver configuration parameters.

Definition at line 8 of file context.h.

### 6.2.2 Member Data Documentation

#### 6.2.2.1 a

```
const DataType Context::a
```

Coefficient for -u_xx.

Definition at line 10 of file context.h.

#### 6.2.2.2 b

```
const DataType Context::b
```

Coefficient for -u_yy.

Definition at line 11 of file context.h.

#### 6.2.2.3 h

```
const DataType Context::h
```

Grid step (duplicated from mesh).

Definition at line 12 of file context.h.

#### 6.2.2.4 max_iter_power

```
const size_t Context::max_iter_power
```

Maximum number of iterations for the power method.

Definition at line 15 of file context.h.

#### 6.2.2.5 max_iter_seidel

```
const size_t Context::max_iter_seidel
```

Maximum number of iterations for the Seidel method.

Definition at line 16 of file context.h.

#### 6.2.2.6 mesh

```
UniformMesh Context::mesh
```

The grid.

Definition at line 9 of file context.h.

#### 6.2.2.7 tol_power

```
const DataType Context::tol_power
```

Tolerance for the power method.

Definition at line 13 of file context.h.

#### 6.2.2.8 tol_seidel

```
const DataType Context::tol_seidel
```

Tolerance for the Seidel method.

Definition at line 14 of file context.h.

The documentation for this struct was generated from the following file:

- context.h

## 6.3 LaplacianOperator Class Reference

Discrete operator: -a u_xx - b u_yy.

```
#include <laplacian_operator.h>
```

**Public Member Functions**

- LaplacianOperator (const UniformMesh &mesh, DataType coeff_x, DataType coeff_y)
  *Constructor.*
- void apply (const std::vector< DataType > &x, std::vector< DataType > &y) const
  *Applies the operator to vector x, result in y.*

### 6.3.1 Detailed Description

Discrete operator: -a u_xx - b u_yy.

Applies the operator to a vector representing function values on the grid.

Definition at line 11 of file laplacian_operator.h.

### 6.3.2 Constructor & Destructor Documentation

#### 6.3.2.1 LaplacianOperator()

```
LaplacianOperator::LaplacianOperator (
            const UniformMesh & mesh,
            DataType coeff_x,
            DataType coeff_y)  [inline]
```

Constructor.

**Parameters**

| | | |
|---|---|---|
| | *mesh* | Grid. |
| | *coeff_x* | Coefficient a. |
| | *coeff_y* | Coefficient b. |

Definition at line 19 of file laplacian_operator.h.

### 6.3.3 Member Function Documentation

#### 6.3.3.1 apply()

```
void LaplacianOperator::apply (
            const std::vector< DataType > & x,
            std::vector< DataType > & y) const  [inline]
```

Applies the operator to vector x, result in y.

**Parameters**

| | | | |
|---|---|---|---|
| | in | *x* | Input vector of size total_nodes(). |
| | out | *y* | Output vector of the same size. |

**Note**

Only interior nodes are computed; boundary values in x are ignored (for eigenvalue problems they should be zero).

Definition at line 30 of file laplacian_operator.h.

The documentation for this class was generated from the following file:

- laplacian_operator.h

## 6.4 OutputHandler Class Reference

Static class for outputting results: printing, saving to binary file, plotting.

```
#include <output_handler.h>
```

**Static Public Member Functions**

- static void print_header (const Context &ctx)

  *Prints solver parameters to console.*
- static void save_full_solution (const Context &ctx, const std::string &filename, const std::vector< DataType > solution)

  *Saves solution vector to a binary file.*
- static void plot_full_solution ()

  *Launches a Python script to visualize the solution.*

### 6.4.1 Detailed Description

Static class for outputting results: printing, saving to binary file, plotting.

Definition at line 11 of file output_handler.h.

### 6.4.2 Member Function Documentation

#### 6.4.2.1 plot_full_solution()

```
void OutputHandler::plot_full_solution () [inline], [static]
```

Launches a Python script to visualize the solution.

Calls "python plot_full.py".

**Exceptions**

| | | |
|---|---|---|
| | *std::runtime_error* | if system call fails. |

Definition at line 65 of file output_handler.h.

#### 6.4.2.2 print_header()

```
void OutputHandler::print_header (
            const Context & ctx) [inline], [static]
```

Prints solver parameters to console.

**Parameters**

| | | |
|---|---|---|
| | *ctx* | Context with parameters. |

Definition at line 17 of file output_handler.h.

#### 6.4.2.3 save_full_solution()

```
void OutputHandler::save_full_solution (
            const Context & ctx,
            const std::string & filename,
            const std::vector< DataType > solution) [inline], [static]
```

Saves solution vector to a binary file.

**Parameters**

| | | |
|---|---|---|
| | *ctx* | Context. |
| | *filename* | Output file name. |
| | *solution* | Solution vector (size total_nodes()). |

**Exceptions**

| | | |
|---|---|---|
| | *std::runtime_error* | if file cannot be opened. |

Format: first two 8-byte integers (nx, ny), then nx∗ny double values in row-major order.

Definition at line 36 of file output_handler.h.

The documentation for this class was generated from the following file:

- output_handler.h

## 6.5 PowerMethod::Params< Operator > Struct Template Reference

Parameters for the power method.

```
#include <power_method.h>
```

**Public Attributes**

- Operator & op

  *Reference to the operator.*
- const UniformMesh & mesh

  *Grid for handling boundaries.*
- std::vector< DataType > & x

  *Initial vector (modified during the process).*
- const DataType tolerance

  *Convergence tolerance.*
- const size_t max_iter

  *Maximum number of iterations.*

### 6.5.1 Detailed Description

**template<typename Operator>**
**struct PowerMethod::Params< Operator >**

Parameters for the power method.

**Template Parameters**

| | | |
|---|---|---|
| | *Operator* | Type of linear operator (must have apply(x,y) method). |

Definition at line 22 of file power_method.h.

### 6.5.2 Member Data Documentation

#### 6.5.2.1 max_iter

```
template<typename Operator>
const size_t PowerMethod::Params< Operator >::max_iter
```

Maximum number of iterations.

Definition at line 27 of file power_method.h.

#### 6.5.2.2 mesh

```
template<typename Operator>
const UniformMesh& PowerMethod::Params< Operator >::mesh
```

Grid for handling boundaries.

Definition at line 24 of file power_method.h.

#### 6.5.2.3 op

```
template<typename Operator>
Operator& PowerMethod::Params< Operator >::op
```

Reference to the operator.

Definition at line 23 of file power_method.h.

#### 6.5.2.4 tolerance

```
template<typename Operator>
const DataType PowerMethod::Params< Operator >::tolerance
```

Convergence tolerance.

Definition at line 26 of file power_method.h.

#### 6.5.2.5 x

```
template<typename Operator>
std::vector<DataType>& PowerMethod::Params< Operator >::x
```

Initial vector (modified during the process).

Definition at line 25 of file power_method.h.

The documentation for this struct was generated from the following file:

- power_method.h

## 6.6 PowerMethod Class Reference

Implements the power method for computing the largest eigenvalue.

```
#include <power_method.h>
```

### Classes

- struct Params

  *Parameters for the power method.*

### Public Member Functions

- template<typename Operator>
  std::tuple< DataType, unsigned > compute_max (Params< Operator > &params) const

  *Computes the largest eigenvalue using the power method.*

### 6.6.1 Detailed Description

Implements the power method for computing the largest eigenvalue.

Definition at line 14 of file power_method.h.

### 6.6.2 Member Function Documentation

#### 6.6.2.1 compute_max()

```
template<typename Operator>
std::tuple< DataType, unsigned > PowerMethod::compute_max (
            Params< Operator > & params) const  [inline]
```

Computes the largest eigenvalue using the power method.

**Parameters**

| | |
|---|---|
| *params* | Parameters, including the initial vector and operator. |

**Returns**

Tuple (eigenvalue, number of iterations).

The initial vector is normalized, then iterations: x_{k+1} = A x_k / ||A x_k||, lambda = x_k^T A x_k. Stopping criterion: residual ||A x - lambda x|| / |lambda| < epsilon.

Definition at line 39 of file power_method.h.

The documentation for this class was generated from the following file:

- power_method.h

## 6.7 SeidelSolver Class Reference

Solves the equation -a u_xx - b u_yy = f using the Seidel method.

```
#include <seidel_solver.h>
```

#### Public Member Functions

- SeidelSolver (const UniformMesh &mesh, DataType coeff_x, DataType coeff_y)

  *Constructor, initializes the right-hand side and boundary conditions.*
- std::vector< DataType > solve (DataType tolerance, size_t max_iter, size_t check_period=50)

  *Performs Seidel iterations.*

### 6.7.1 Detailed Description

Solves the equation -a u_xx - b u_yy = f using the Seidel method.

Definition at line 11 of file seidel_solver.h.

### 6.7.2 Constructor & Destructor Documentation

#### 6.7.2.1 SeidelSolver()

```
SeidelSolver::SeidelSolver (
            const UniformMesh & mesh,
            DataType coeff_x,
            DataType coeff_y)  [inline]
```

Constructor, initializes the right-hand side and boundary conditions.

**Parameters**

| | | |
|---|---|---|
| | *mesh* | Grid. |
| | *coeff_x* | Coefficient a. |
| | *coeff_y* | Coefficient b. |

Definition at line 19 of file seidel_solver.h.

### 6.7.3 Member Function Documentation

#### 6.7.3.1 solve()

```
std::vector< DataType > SeidelSolver::solve (
            DataType tolerance,
            size_t max_iter,
            size_t check_period = 50)  [inline]
```

Performs Seidel iterations.

**Parameters**

| | | |
|---|---|---|
| | *tolerance* | Tolerance for residual reduction. |
| | *max_iter* | Maximum number of iterations. |
| | *check_period* | Period for residual check (default 50). |

**Returns**

Solution vector (size total_nodes()).

The residual is computed as sqrt(sum (Lu - f)^2) over interior nodes.

Definition at line 34 of file seidel_solver.h.

The documentation for this class was generated from the following file:

- seidel_solver.h

## 6.8 ShiftedPowerMethod Class Reference

Computes the smallest eigenvalue using the shifted power method.

```
#include <shifted_power_method.h>
```

### Public Types

- template<typename Operator>
  using Params = PowerMethod::Params<Operator>

  *Synonim for Params in PowerMethod.*

### Public Member Functions

- ShiftedPowerMethod (DataType _shift)

  *Constructor with a given shift (usually the largest eigenvalue).*
- template<typename Operator>
  std::tuple< DataType, unsigned > compute_min (Params< Operator > &params) const

  *Computes the smallest eigenvalue.*

### 6.8.1 Detailed Description

Computes the smallest eigenvalue using the shifted power method.

Applies the power method to the operator shift∗I - A.

Definition at line 10 of file shifted_power_method.h.

## 6.8.2 Member Typedef Documentation

### 6.8.2.1 Params

```
template<typename Operator>
using ShiftedPowerMethod::Params = PowerMethod::Params<Operator>
```

Synonim for Params in PowerMethod.

Definition at line 16 of file shifted_power_method.h.

## 6.8.3 Constructor & Destructor Documentation

### 6.8.3.1 ShiftedPowerMethod()

```
ShiftedPowerMethod::ShiftedPowerMethod (
            DataType _shift)  [inline]
```

Constructor with a given shift (usually the largest eigenvalue).

**Parameters**

| | | |
|---|---|---|
| | *_shift* | Shift value. |

Definition at line 22 of file shifted_power_method.h.

## 6.8.4 Member Function Documentation

### 6.8.4.1 compute_min()

```
template<typename Operator>
std::tuple< DataType, unsigned > ShiftedPowerMethod::compute_min (
            Params< Operator > & params) const  [inline]
```

Computes the smallest eigenvalue.

**Parameters**

| | | |
|---|---|---|
| | *params* | Parameters for the power method (initial vector, etc.). |

**Returns**

Tuple (eigenvalue, number of iterations).

Uses the shifted operator (shift∗I - A) and then lambda_min = shift - mu, where mu is the largest eigenvalue of the shifted operator. Internal helper class implementing the shifted operator.

Applies the shifted operator.

Definition at line 32 of file shifted_power_method.h.

The documentation for this class was generated from the following file:

- shifted_power_method.h

## 6.9 SolverController Class Reference

Controller managing the solution of the equation, eigenvalue computation, and output/saving of results.

```
#include <solver_controller.h>
```

### Public Member Functions

- SolverController (double a, double b, double h, double tol_power, double tol_seidel, int max_iter_power, int max_iter_seidel)

  *Constructor.*
- void run_all (InitialVectorType type_max=InitialVectorType::UnitConstant, InitialVectorType type_min=InitialVectorType::UnitConstant)

  *Runs the Seidel method, saves/plots the solution, then computes eigenvalues.*
- void run_seidel ()

  *Runs the Seidel method with saving and visualization of the solution.*
- void run_eigen (InitialVectorType type_max=InitialVectorType::UnitConstant, InitialVectorType type_min=InitialVectorType::UnitConstant)

  *Runs the eigenvalue computation.*

### 6.9.1 Detailed Description

Controller managing the solution of the equation, eigenvalue computation, and output/saving of results.

Definition at line 17 of file solver_controller.h.

### 6.9.2 Constructor & Destructor Documentation

#### 6.9.2.1 SolverController()

```
SolverController::SolverController (
            double a,
            double b,
            double h,
            double tol_power,
            double tol_seidel,
            int max_iter_power,
            int max_iter_seidel)  [inline]
```

Constructor.

**Parameters**

| | |
|---|---|
| *a* | Coefficient a. |
| *b* | Coefficient b. |
| *h* | Grid step. |
| *tol_power* | Tolerance for the power method. |
| *tol_seidel* | Tolerance for the Seidel method. |
| *max_iter_power* | Maximum number of iterations for the power method. |
| *max_iter_seidel* | Maximum number of iterations for the Seidel method. |

Definition at line 29 of file solver_controller.h.

### 6.9.3 Member Function Documentation

#### 6.9.3.1 run_all()

```
void SolverController::run_all (
            InitialVectorType type_max = InitialVectorType::UnitConstant,
            InitialVectorType type_min = InitialVectorType::UnitConstant)  [inline]
```

Runs the Seidel method, saves/plots the solution, then computes eigenvalues.

**Parameters**

| | |
|---|---|
| *type_max* | Type of initial vector for the maximum eigenvalue. |
| *type_min* | Type of initial vector for the minimum eigenvalue. |

Definition at line 40 of file solver_controller.h.

#### 6.9.3.2 run_eigen()

```
void SolverController::run_eigen (
            InitialVectorType type_max = InitialVectorType::UnitConstant,
            InitialVectorType type_min = InitialVectorType::UnitConstant)  [inline]
```

Runs the eigenvalue computation.

**Parameters**

| | |
|---|---|
| *type_max* | Type of initial vector for the maximum eigenvalue. |
| *type_min* | Type of initial vector for the minimum eigenvalue. |

Definition at line 64 of file solver_controller.h.

#### 6.9.3.3 run_seidel()

```
void SolverController::run_seidel ()  [inline]
```

Runs the Seidel method with saving and visualization of the solution.

Definition at line 52 of file solver_controller.h.

The documentation for this class was generated from the following file:

- solver_controller.h

## 6.10 UniformMesh Class Reference

Represents a uniform rectangular grid on [0,1]x[0,1] with a given step.

```
#include <mesh.h>
```

**Public Types**

- using IndexIterationFunction = std::function<void(size_t, size_t, size_t)>

**Public Member Functions**

- UniformMesh (DataType step)

  *Constructor with desired step.*
- std::size_t nodes_per_side () const

  *Returns the number of nodes per side.*
- DataType step () const

  *Returns the grid step.*
- std::size_t full_index (std::size_t i, std::size_t j) const

  *Computes the linear index of node (i,j).*
- bool is_inner (std::size_t i, std::size_t j) const

  *Checks whether a node is interior (not on the boundary).*
- std::size_t inner_nodes () const

  *Returns the total number of interior nodes.*
- std::size_t total_nodes () const

  *Returns the total number of nodes, including boundary.*
- DataType x (std::size_t i) const

  *Returns x-coordinate of node i.*
- DataType y (std::size_t j) const

  *Returns y-coordinate of node j.*
- void for_each_inner (const IndexIterationFunction &f) const

  *Applies a function to all interior nodes.*

## 6.10.1 Detailed Description

Represents a uniform rectangular grid on [0,1]x[0,1] with a given step.

The grid is square, nodes at points (i∗h, j∗h) where i,j = 0,...,N-1, h  actual step, N = 1/h + 1.

Definition at line 13 of file mesh.h.

## 6.10.2 Member Typedef Documentation

### 6.10.2.1 IndexIterationFunction

```
using UniformMesh::IndexIterationFunction = std::function<void(size_t, size_t, size_t)>
```

Definition at line 91 of file mesh.h.

## 6.10.3 Constructor & Destructor Documentation

### 6.10.3.1 UniformMesh()

```
UniformMesh::UniformMesh (
            DataType step)  [inline], [explicit]
```

Constructor with desired step.

**Parameters**

| | | |
|---|---|---|
| | *step* | Desired step. Must be in (0, 0.5]. The step is adjusted so that the number of intervals is integer: step = 1/(N-1). |

**Exceptions**

| | | |
|---|---|---|
| | *std::invalid_argument* | if step $<=$ 0 or step $>$ 0.5. |

Definition at line 21 of file mesh.h.

## 6.10.4 Member Function Documentation

### 6.10.4.1 for_each_inner()

```
void UniformMesh::for_each_inner (
            const IndexIterationFunction & f) const  [inline]
```

Applies a function to all interior nodes.

**Parameters**

| | | |
|---|---|---|
| | *f* | Function taking (i, j, linear_index). |

**Note**

The function is called for each interior node in row-major order.

Definition at line 98 of file mesh.h.

### 6.10.4.2 full_index()

```
std::size_t UniformMesh::full_index (
            std::size_t i,
            std::size_t j) const  [inline]
```

Computes the linear index of node (i,j).

**Parameters**

| | | |
|---|---|---|
| | *i* | Index along x (0-based). |
| | *j* | Index along y (0-based). |

**Returns**

Linear index = i + j $*$ nodes_per_side().

Definition at line 46 of file mesh.h.

#### 6.10.4.3 inner_nodes()

```
std::size_t UniformMesh::inner_nodes () const [inline]
```

Returns the total number of interior nodes.

**Returns**

(N-2)∗(N-2).

Definition at line 65 of file mesh.h.

#### 6.10.4.4 is_inner()

```
bool UniformMesh::is_inner (
            std::size_t i,
            std::size_t j) const [inline]
```

Checks whether a node is interior (not on the boundary).

**Parameters**

| | | |
|---|---|---|
| | *i* | Index along x. |
| | *j* | Index along y. |

**Returns**

true if 0 < i < N-1 and 0 < j < N-1.

Definition at line 56 of file mesh.h.

#### 6.10.4.5 nodes_per_side()

```
std::size_t UniformMesh::nodes_per_side () const [inline]
```

Returns the number of nodes per side.

**Returns**

Number of nodes N.

Definition at line 32 of file mesh.h.

#### 6.10.4.6 step()

```
DataType UniformMesh::step () const [inline]
```

Returns the grid step.

**Returns**

Step h.

Definition at line 38 of file mesh.h.

#### 6.10.4.7 total_nodes()

```
std::size_t UniformMesh::total_nodes () const [inline]
```

Returns the total number of nodes, including boundary.

**Returns**

N∗N.

Definition at line 74 of file mesh.h.

#### 6.10.4.8 x()

```
DataType UniformMesh::x (
            std::size_t i) const [inline]
```

Returns x-coordinate of node i.

**Parameters**

| | | |
|---|---|---|
| | *i* | Index along x. |

**Returns**

i ∗ step.

Definition at line 83 of file mesh.h.

#### 6.10.4.9 y()

```
DataType UniformMesh::y (
            std::size_t j) const [inline]
```

Returns y-coordinate of node j.

**Parameters**

| | | |
|---|---|---|
| | *j* | Index along y. |

**Returns**

j ∗ step.

Definition at line 89 of file mesh.h.

The documentation for this class was generated from the following file:

- mesh.h

# Chapter 7

# File Documentation

## 7.1 clock_wrapper.h File Reference

```
#include "context.h"
#include <chrono>
```

### Classes

- class Clock

  *Simple RAII timer for measuring elapsed time.*

## 7.2 clock_wrapper.h

Go to the documentation of this file.

```
#pragma once
#include "context.h"
#include <chrono>

class Clock {
public:
    using TimePoint = std::chrono::steady_clock::time_point;
    using ChronoClock = std::chrono::high_resolution_clock;
    using Elapsed = std::chrono::duration<DataType>;

    Clock() : m_start(ChronoClock::now()) { }

    Elapsed elapsed() {
        auto end = ChronoClock::now();
        return end - m_start;
    }

    Elapsed restart() {
        Elapsed elapsed = std::move(this->elapsed());
        m_start = ChronoClock::now();
        return elapsed;
    }

private:
    TimePoint m_start;
};
```

## 7.3 context.h File Reference

```
#include "mesh.h"
```

### Classes

- struct Context

  *Contains all solver configuration parameters.*

## 7.4 context.h

Go to the documentation of this file.

```
00001 #pragma once
00002 #include "mesh.h"
00003
00008 struct Context {
00009     UniformMesh mesh;
00010     const DataType a;
00011     const DataType b;
00012     const DataType h;
00013     const DataType tol_power;
00014     const DataType tol_seidel;
00015     const size_t max_iter_power;
00016     const size_t max_iter_seidel;
00017 };
```

## 7.5 eigen_theory.h File Reference

```
#include "context.h"
#include "math_utils.h"
#include <cmath>
#include <tuple>
```

### Namespaces

- namespace eigen_theory

  *Theoretical formulas for eigenvalues of the Laplace operator.*

### Functions

- DataType eigen_theory::lambda_min (DataType coeff_x, DataType coeff_y, DataType h)

  *Computes the theoretical minimum eigenvalue.*
- DataType eigen_theory::lambda_max (DataType coeff_x, DataType coeff_y, DataType h)

  *Computes the theoretical maximum eigenvalue.*
- std::tuple< DataType, DataType > eigen_theory::lambda_max_min (DataType coeff_x, DataType coeff_y, DataType h)

  *Returns both eigenvalues as a tuple.*

## 7.6 eigen_theory.h

Go to the documentation of this file.

```
#pragma once
#include "context.h"
#include "math_utils.h"
#include <cmath>
#include <tuple>

namespace eigen_theory {
    inline DataType lambda_min(DataType coeff_x, DataType coeff_y, DataType h) {
        DataType coeff_sum = coeff_x + coeff_y;
        DataType factor = 4.0 * coeff_sum / (h * h);
        DataType sin_term = std::sin(math_utils::PI * h / 2.0);
        return factor * sin_term * sin_term;
    }

    inline DataType lambda_max(DataType coeff_x, DataType coeff_y, DataType h) {
        DataType coeff_sum = coeff_x + coeff_y;
        DataType factor = 4.0 * coeff_sum / (h * h);
        DataType cos_term = std::cos(math_utils::PI * h / 2.0);
        return factor * cos_term * cos_term;
    }

    inline std::tuple<DataType, DataType> lambda_max_min(DataType coeff_x, DataType coeff_y, DataType
h) {
        return {
            lambda_max(coeff_x, coeff_y, h),
            lambda_min(coeff_x, coeff_y, h)
        };
    }
}
```

## 7.7 functions.h File Reference

```
#include "context.h"
#include <cmath>
```

### Namespaces

- namespace exact

  *Exact solution and right-hand side for the test equation.*

### Functions

- DataType exact::u (DataType x, DataType y)

  *Exact solution u(x,y) = x^2 + cos^2(y∗x).*
- DataType exact::source (DataType x, DataType y, DataType coeff_x, DataType coeff_y)

  *Right-hand side corresponding to the exact solution.*
- DataType exact::boundary (DataType x, DataType y)

  *Boundary condition (same as exact solution).*

## 7.8 functions.h

Go to the documentation of this file.

```
#pragma once
#include "context.h"
#include <cmath>

namespace exact {
    inline DataType u(DataType x, DataType y) {
        return x * x + std::cos(y * x) * std::cos(y * x);
    }

    inline DataType source(DataType x, DataType y, DataType coeff_x, DataType coeff_y) {
        DataType yx = y * x;
        DataType cos2yx = std::cos(2.0 * yx);
        return -2.0 * coeff_x + 2.0 * cos2yx * (coeff_x * y * y + coeff_y * x * x);
    }

    inline DataType boundary(DataType x, DataType y) {
        return u(x, y);
    }
}
```

## 7.9 initial_vector_type.h File Reference

### Enumerations

- enum class InitialVectorType { Random , UnitConstant , Alternating }

  *Type of initial vector for eigenvalue iterations.*

### 7.9.1 Enumeration Type Documentation

#### 7.9.1.1 InitialVectorType

```
enum class InitialVectorType  [strong]
```

Type of initial vector for eigenvalue iterations.

**Enumerator**

| | |
|---|---|
| Random | Random values in [-1,1]. |
| UnitConstant | All interior nodes equal to 1. |
| Alternating | Alternating sign (-1)^(i+j). |

Definition at line 7 of file initial_vector_type.h.

## 7.10 initial_vector_type.h

Go to the documentation of this file.

```
#pragma once

enum class InitialVectorType {
    Random,
    UnitConstant,
    Alternating
};
```

## 7.11 laplacian_operator.h File Reference

```
#include "mesh.h"
#include <vector>
#include <omp.h>
```

### Classes

- class LaplacianOperator

  *Discrete operator: -a u_xx - b u_yy.*

## 7.12 laplacian_operator.h

Go to the documentation of this file.

```
00001 #pragma once
00002 #include "mesh.h"
00003 #include <vector>
00004 #include <omp.h>
00005
00011 class LaplacianOperator {
00012 public:
00019     LaplacianOperator(const UniformMesh& mesh, DataType coeff_x, DataType coeff_y)
00020         : m_mesh(mesh), m_coeff_x(coeff_x), m_coeff_y(coeff_y) {
00021     }
00022
00030     void apply(const std::vector<DataType>& x, std::vector<DataType>& y) const {
00031         const DataType inv_h2 = 1.0 / (m_mesh.step() * m_mesh.step());
00032         const std::size_t n = m_mesh.nodes_per_side();
00033         const std::size_t stride = n;
00034
00035 #ifdef _OPENMP
00036 #pragma omp parallel for
00037 #endif
00038         for (std::ptrdiff_t j = 1; j < static_cast<std::ptrdiff_t>(n - 1); ++j) {
00039             for (std::size_t i = 1; i < n - 1; ++i) {
00040                 std::size_t idx = i + j * stride;
00041                 DataType center = x[idx];
00042                 DataType left = x[idx - 1];
00043                 DataType right = x[idx + 1];
00044                 DataType down = x[idx - stride];
00045                 DataType up = x[idx + stride];
00046
00047                 DataType d2x = -m_coeff_x * inv_h2 * (left - 2.0 * center + right);
00048                 DataType d2y = -m_coeff_y * inv_h2 * (down - 2.0 * center + up);
00049
00050                 y[idx] = d2x + d2y;
00051             }
00052         }
00053     }
00054
00055 private:
00056     const UniformMesh& m_mesh;
00057     DataType m_coeff_x, m_coeff_y;
00058 };
```

## 7.13 main.cpp File Reference

```
#include "solver_controller.h"
```

**Functions**

- int main ()

  *Entry point.*

### 7.13.1 Function Documentation

#### 7.13.1.1 main()

```
int main ()
```

Entry point.

**Returns**

0 on success, non-zero otherwise.

Task variant: 12. Uses parameters: a=0.9, b=1.1, h=0.01, tolerance=1e-6, max iterations=100000. Initial vectors: Alternating for maximum, UnitConstant for minimum.

Definition at line 10 of file main.cpp.

## 7.14 main.cpp

Go to the documentation of this file.

```
00001 #include "solver_controller.h"
00002
00010 int main() {
00011     const DataType a = 0.9, b = 1.1;
00012     const DataType h = 0.1;
00013     const DataType tol_power = 1e-6, tol_seidel = 1e-6;
00014     const size_t max_iter_power = 100000, max_iter_seidel = 100000;
00015
00016     try {
00017
00018         SolverController solver(a, b, h, tol_power, tol_seidel,
00019             max_iter_power, max_iter_seidel);
00020         solver.run_all(
00021             InitialVectorType::Alternating,
00022             InitialVectorType::UnitConstant
00023         );
00024
00025     }
00026     catch (const std::exception& e) {
00027         std::cerr « "Error: " « e.what() « "\n";
00028         return EXIT_FAILURE;
00029     }
00030     return EXIT_SUCCESS;
00031 }
```

## 7.15 mainpage.dox File Reference

## 7.16 math_utils.h File Reference

```
#include "context.h"
#include <vector>
#include <cmath>
#include <omp.h>
```

### Namespaces

- namespace math_utils

  *Auxiliary mathematical functions: dot product, norm, constants.*

### Functions

- DataType math_utils::dot (const std::vector< DataType > &a, const std::vector< DataType > &b)

  *Dot product of two vectors.*
- DataType math_utils::norm (const std::vector< DataType > &v)

  *Euclidean norm of a vector.*

### Variables

- constexpr long double math_utils::PI = 3.14159265358979323846

  *Pi constant.*

## 7.17 math_utils.h

Go to the documentation of this file.

```
#pragma once
#include "context.h"
#include <vector>
#include <cmath>
#include <omp.h>

namespace math_utils {
    constexpr long double PI = 3.14159265358979323846;

    inline DataType dot(const std::vector<DataType>& a, const std::vector<DataType>& b) {
        DataType sum = 0.0;
#pragma omp parallel for reduction(+:sum)
        for (std::ptrdiff_t i = 0; i < static_cast<std::ptrdiff_t>(a.size()); ++i) {
            sum += a[i] * b[i];
        }
        return sum;
    }

    inline DataType norm(const std::vector<DataType>& v) {
        return std::sqrt(dot(v, v));
    }

} // namespace math_utils
```

## 7.18 mesh.h File Reference

```
#include <stdexcept>
#include <functional>
```

### Classes

- class UniformMesh

  *Represents a uniform rectangular grid on [0,1]x[0,1] with a given step.*

### Typedefs

- using DataType = double

## 7.18.1 Typedef Documentation

### 7.18.1.1 DataType

```
using DataType = double
```

Definition at line 5 of file mesh.h.

# 7.19 mesh.h

Go to the documentation of this file.

```
00001 #pragma once
00002 #include <stdexcept>
00003 #include <functional>
00004
00005 using DataType = double;
00006
00013 class UniformMesh {
00014 public:
00021     explicit UniformMesh(DataType step) : m_step(step) {
00022         if (step <= 0.0 || step > 0.5)
00023             throw std::invalid_argument("Step must be in (0, 0.5]");
00024         m_nodes_per_side = static_cast<std::size_t>(1.0 / step) + 1;
00025         m_step = 1.0 / (m_nodes_per_side - 1);
00026     }
00027
00032     std::size_t nodes_per_side() const { return m_nodes_per_side; }
00033
00038     DataType step() const { return m_step; }
00039
00046     std::size_t full_index(std::size_t i, std::size_t j) const {
00047         return i + j * m_nodes_per_side;
00048     }
00049
00056     bool is_inner(std::size_t i, std::size_t j) const {
00057         return i > 0 && i < m_nodes_per_side - 1 &&
00058             j > 0 && j < m_nodes_per_side - 1;
00059     }
00060
00065     std::size_t inner_nodes() const {
00066         std::size_t n = m_nodes_per_side - 2;
00067         return n * n;
00068     }
00069
00074     std::size_t total_nodes() const {
00075         return m_nodes_per_side * m_nodes_per_side;
00076     }
00077
00083     DataType x(std::size_t i) const { return i * m_step; }
00089     DataType y(std::size_t j) const { return j * m_step; }
00090
00091     using IndexIterationFunction = std::function<void(size_t, size_t, size_t)>;
00092
00098     void for_each_inner(const IndexIterationFunction& f) const {
00099         for (std::size_t j = 1; j < m_nodes_per_side - 1; ++j)
00100             for (std::size_t i = 1; i < m_nodes_per_side - 1; ++i)
00101                 f(i, j, full_index(i, j));
00102     }
00103
00104 private:
00105     DataType m_step;
00106     std::size_t m_nodes_per_side;
00107 };
```

## 7.20 output_handler.h File Reference

```
#include "context.h"
#include <fstream>
#include <iostream>
#include <iomanip>
```

### Classes

- class OutputHandler

  *Static class for outputting results: printing, saving to binary file, plotting.*

## 7.21 output_handler.h

Go to the documentation of this file.

```
#pragma once
#include "context.h"
#include <fstream>
#include <iostream>
#include <iomanip>

class OutputHandler {
public:
    static void print_header(const Context& ctx) {
        std::cout « std::fixed « std::setprecision(12);
        std::cout « "========================================\n";
        std::cout « "Solving -a u_xx - b u_yy = f on [0,1]^2\n";
        std::cout « "a = " « ctx.a « ", b = " « ctx.b « "\n";
        std::cout « "h = " « ctx.h « ", h^2 = " « ctx.h * ctx.h « "\n";
        std::cout « "Tolerance (power) = " « ctx.tol_power « ", max iterations = " « ctx.max_iter_power « "\n";
        std::cout « "Tolerance (seidel) = " « ctx.tol_seidel « ", max iterations = " « ctx.max_iter_seidel « "\n";
        std::cout « "========================================\n";
    }

    static void save_full_solution(
        const Context& ctx,
        const std::string& filename,
        const std::vector<DataType> solution
    ) {
        std::size_t nx = ctx.mesh.nodes_per_side();
        std::size_t ny = ctx.mesh.nodes_per_side();
        std::ofstream out(filename, std::ios::binary);
        if (!out) {
            throw std::runtime_error("Error: cannot open file " + filename + " for writing.\n");
        }
        out.write(reinterpret_cast<const char*>(&nx), sizeof(nx));
        out.write(reinterpret_cast<const char*>(&ny), sizeof(ny));
        for (std::size_t j = 0; j < ny; ++j) {
            for (std::size_t i = 0; i < nx; ++i) {
                double val = solution[ctx.mesh.full_index(i, j)];
                out.write(reinterpret_cast<const char*>(&val), sizeof(val));
            }
        }
        out.close();
        std::cout « "Full solution saved to " « filename « " ("
            « nx « "x" « ny « " points)\n";
    }

    static void plot_full_solution() {
        std::cout « "Launching Python script to plot full solution...\n";
        int ret = std::system("python plot_full.py");
        if (ret != 0) {
            throw std::runtime_error("Warning: Python script execution failed (code " + std::to_string(ret) + ").\n");
        }
    }
};
```

## 7.22 plot_full.py File Reference

( )

### Namespaces

- namespace plot_full

### Functions

- np.ndarray plot_full.u_exact (Union[float, np.ndarray] x, Union[float, np.ndarray] y)

  *Exact analytical solution of the problem.*
- Tuple[np.ndarray, np.ndarray, np.ndarray] plot_full.read_full_solution (str filename)

  *Reads a binary solution file created by OutputHandler.save_full_solution.*
- None plot_full.main ()

  *Main function of the script.*

### 7.22.1 Detailed Description

( )

Definition in file plot_full.py.

## 7.23 plot_full.py

Go to the documentation of this file.

```
# -*- coding: utf-8 -*-


"""!
@file plot_full.py
@brief Visualization of the numerical solution and comparison with the exact one.
@details The script reads a binary file solution.bin containing the grid and numerical solution values,
         creates 3D plots of the exact solution, numerical solution, and absolute error, as well as a 2D error map.
         Uses matplotlib and numpy.
"""

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import struct
import sys

def u_exact(x: Union[float, np.ndarray], y: Union[float, np.ndarray]) -> np.ndarray:
    """!
    @brief Exact analytical solution of the problem.
    @param x x-coordinate (scalar or numpy array).
    @param y y-coordinate (scalar or numpy array).
    @return u(x,y) = x^2 + cos^2(y*x).
    """
    return x**2 + np.cos(y * x)**2

def read_full_solution(filename: str) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """!
    @brief Reads a binary solution file created by OutputHandler::save_full_solution.
    @param filename File name (e.g., 'solution.bin').
    @return Tuple (X, Y, u_num), where X, Y are coordinate matrices (meshgrid),
            u_num is a 2D array of the numerical solution.
    @throws ValueError if the file is too short or has an invalid format.
```

```
    @details File format:
             - first 8 bytes: number of nodes along x (nx, size_t)
             - next 8 bytes: number of nodes along y (ny, size_t)
             - then nx*ny double values in row-major order (j=0..ny-1, i=0..nx-1)
    """
    with open(filename, 'rb') as f:
        nx_bytes = f.read(8)
        ny_bytes = f.read(8)
        if len(nx_bytes) < 8 or len(ny_bytes) < 8:
            raise ValueError("File too short: cannot read dimensions")
        nx = struct.unpack('Q', nx_bytes)[0]
        ny = struct.unpack('Q', ny_bytes)[0]

        total = nx * ny
        data_bytes = f.read(total * 8)
        if len(data_bytes) < total * 8:
            raise ValueError("File too short: cannot read all data")

        data = struct.unpack('d' * total, data_bytes)

    u_num = np.array(data).reshape((ny, nx))
    x = np.linspace(0, 1, nx)
    y = np.linspace(0, 1, ny)
    X, Y = np.meshgrid(x, y)
    return X, Y, u_num

def main() -> None:
    """!
    @brief Main function of the script.
    @details Performs reading of the file, computation of exact solution and error,
             creates and saves plots (comparison.png, error_map.png),
             and displays them on the screen.
    """
    try:
        X, Y, u_num = read_full_solution('solution.bin')
    except Exception as e:
        print(f"Error reading binary file: {e}")
        sys.exit(1)

    u_ex = u_exact(X, Y)
    error = np.abs(u_ex - u_num)

    fig = plt.figure(figsize=(18, 5))

    # Exact solution
    ax1 = fig.add_subplot(131, projection='3d')
    surf1 = ax1.plot_surface(X, Y, u_ex, cmap='viridis', linewidth=0, antialiased=True)
    ax1.set_title('Exact solution')
    ax1.set_xlabel('x')
    ax1.set_ylabel('y')
    ax1.set_zlabel('u')
    fig.colorbar(surf1, ax=ax1, shrink=0.5, aspect=10)

    # Numerical solution
    ax2 = fig.add_subplot(132, projection='3d')
    surf2 = ax2.plot_surface(X, Y, u_num, cmap='viridis', linewidth=0, antialiased=True)
    ax2.set_title('Numerical solution')
    ax2.set_xlabel('x')
    ax2.set_ylabel('y')
    ax2.set_zlabel('u')
    fig.colorbar(surf2, ax=ax2, shrink=0.5, aspect=10)

    # Absolute error
    ax3 = fig.add_subplot(133, projection='3d')
    surf3 = ax3.plot_surface(X, Y, error, cmap='hot', linewidth=0, antialiased=True)
    ax3.set_title('Absolute error')
    ax3.set_xlabel('x')
    ax3.set_ylabel('y')
    ax3.set_zlabel('error')
    fig.colorbar(surf3, ax=ax3, shrink=0.5, aspect=10)

    plt.tight_layout()
    plt.savefig('comparison.png', dpi=300, bbox_inches='tight')
    print("Comparison plot saved as comparison.png")

    # Error map (2D)
    plt.figure(figsize=(8, 6))
    plt.pcolormesh(X, Y, error, shading='auto', cmap='hot')
    plt.colorbar(label='Absolute error')
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('Error map |exact - numerical|')
    plt.axis('equal')
    plt.tight_layout()
    plt.savefig('error_map.png', dpi=300)
    print("Error map saved as error_map.png")

```

```
    plt.show()

if __name__ == "__main__":
    main()
```

## 7.24 power_method.h File Reference

```
#include "mesh.h"
#include "math_utils.h"
#include "context.h"
#include <vector>
#include <cmath>
#include <iostream>
#include <tuple>
```

### Classes

- class PowerMethod

  *Implements the power method for computing the largest eigenvalue.*
- struct PowerMethod::Params< Operator >

  *Parameters for the power method.*

## 7.25 power_method.h

Go to the documentation of this file.

```
#pragma once
#include "mesh.h"
#include "math_utils.h"
#include "context.h"
#include <vector>
#include <cmath>
#include <iostream>
#include <tuple>

class PowerMethod {
public:
    template<typename Operator>
    struct Params {
        Operator& op;
        const UniformMesh& mesh;
        std::vector<DataType>& x;
        const DataType tolerance;
        const size_t max_iter;
    };

    template<typename Operator>
    std::tuple<DataType, unsigned> compute_max(Params<Operator>& params) const {
        const size_t N = params.x.size();
        const size_t stride = params.mesh.nodes_per_side();

        normalize_with_boundary(params.x, params.mesh);

        DataType lambda = 0.0;
        std::vector<DataType> y(N, 0.0);
        unsigned iterations = 0;

        for (size_t iter = 0; iter < params.max_iter; ++iter) {
            ++iterations;
            params.op.apply(params.x, y);

            lambda = math_utils::dot(params.x, y);

            DataType residual = 0.0;
#ifdef _OPENMP
```

```
#pragma omp parallel for reduction(+:residual)
#endif
            for (std::ptrdiff_t i = 0; i < static_cast<std::ptrdiff_t>(N); ++i) {
                DataType r = y[i] - lambda * params.x[i];
                residual += r * r;
            }
            residual = std::sqrt(residual);

            if (std::abs(lambda) > 0
                && residual / std::abs(lambda) < params.tolerance) {
                break;
            }

            DataType y_norm = math_utils::norm(y);
            if (y_norm == 0) break;
            // x_{k+1} = x_k / ||x_k||
#ifdef _OPENMP
#pragma omp parallel for
#endif
            for (std::ptrdiff_t i = 0; i < static_cast<std::ptrdiff_t>(N); ++i) {
                params.x[i] = y[i] / y_norm;
            }
            enforce_zero_boundary(params.x, params.mesh);
        }
        return { lambda, iterations };
    }

private:
    static void normalize_with_boundary(std::vector<DataType>& v, const UniformMesh& mesh) {
        DataType n = 0.0;
        mesh.for_each_inner(
            [&](std::size_t, std::size_t, std::size_t idx) {
                n += v[idx] * v[idx];
            }
        );
        n = std::sqrt(n);
        if (n > 0) {
            mesh.for_each_inner(
                [&](std::size_t, std::size_t, std::size_t idx) {
                    v[idx] /= n;
                }
            );
        }
        else {
            DataType val = 1.0 / std::sqrt(mesh.inner_nodes());
            mesh.for_each_inner(
                [&](std::size_t, std::size_t, std::size_t idx) {
                    v[idx] = val;
                }
            );
        }
        enforce_zero_boundary(v, mesh);
    }

    static void enforce_zero_boundary(std::vector<DataType>& v, const UniformMesh& mesh) {
        std::size_t n = mesh.nodes_per_side();
#ifdef _OPENMP
#pragma omp parallel for
#endif
        for (std::ptrdiff_t j = 0; j < static_cast<std::ptrdiff_t>(n); ++j) {
            v[mesh.full_index(0, j)] = 0.0;
            v[mesh.full_index(n - 1, j)] = 0.0;
        }
#ifdef _OPENMP
#pragma omp parallel for
#endif
        for (std::ptrdiff_t i = 1; i < static_cast<std::ptrdiff_t>(n - 1); ++i) {
            v[mesh.full_index(i, 0)] = 0.0;
            v[mesh.full_index(i, n - 1)] = 0.0;
        }
    }
};
```

## 7.26 seidel_solver.h File Reference

```
#include "mesh.h"
#include <vector>
#include <cmath>
#include <omp.h>
```

### Classes

- class SeidelSolver

  *Solves the equation -a u_xx - b u_yy = f using the Seidel method.*

## 7.27 seidel_solver.h

Go to the documentation of this file.

```
#pragma once
#include "mesh.h"
#include <vector>
#include <cmath>
#include <omp.h>

class SeidelSolver {
public:
    SeidelSolver(const UniformMesh& mesh, DataType coeff_x, DataType coeff_y)
        : m_mesh(mesh), m_coeff_x(coeff_x), m_coeff_y(coeff_y),
        m_right_hand_side(m_mesh.total_nodes(), 0.0),
        m_solution(m_mesh.total_nodes(), 0.0) {
        init();
    }

    std::vector<DataType> solve(DataType tolerance, size_t max_iter, size_t check_period = 50) {
        const DataType h2 = m_mesh.step() * m_mesh.step();
        const DataType coeff_sum = 2.0 * (m_coeff_x + m_coeff_y);
        const size_t stride = m_mesh.nodes_per_side();

        auto residual_norm = [&]() -> DataType {
            DataType res = 0.0;
#ifdef _OPENMP
#pragma omp parallel for reduction(+:res)
#endif
            for (std::ptrdiff_t j = 1; j < static_cast<std::ptrdiff_t>(stride - 1); ++j) {
                for (size_t i = 1; i < stride - 1; ++i) {
                    size_t idx = i + j * stride;
                    DataType center = m_solution[idx];
                    DataType left = m_solution[idx - 1];
                    DataType right = m_solution[idx + 1];
                    DataType down = m_solution[idx - stride];
                    DataType up = m_solution[idx + stride];

                    DataType d2x = -m_coeff_x * (left - 2.0 * center + right) / h2;
                    DataType d2y = -m_coeff_y * (down - 2.0 * center + up) / h2;
                    DataType Lu = d2x + d2y;
                    DataType r = Lu - m_right_hand_side[idx];
                    res += r * r;
                }
            }
            return std::sqrt(res);
        };

        DataType initial_residual = residual_norm();
        DataType target_residual = tolerance * initial_residual;

        for (size_t iter = 0; iter < max_iter; ++iter) {
            for (size_t j = 1; j < stride - 1; ++j) {
                for (size_t i = 1; i < stride - 1; ++i) {
                    size_t idx = i + j * stride;
                    DataType numerator =
                        m_coeff_x * (m_solution[idx - 1] + m_solution[idx + 1]) +
                        m_coeff_y * (m_solution[idx - stride] + m_solution[idx + stride]) +
                        m_right_hand_side[idx] * h2;
                    m_solution[idx] = numerator / coeff_sum;
                }
            }

            if (iter % check_period == 0) {
                DataType current_residual = residual_norm();
                if (current_residual < target_residual)
                    break;
            }
        }

        return m_solution;
    }

private:
    void init() {
```

```
        for (size_t j = 0; j < m_mesh.nodes_per_side(); ++j) {
            for (size_t i = 0; i < m_mesh.nodes_per_side(); ++i) {
                DataType x = m_mesh.x(i);
                DataType y = m_mesh.y(j);
                m_right_hand_side[m_mesh.full_index(i, j)]
                    = exact::source(x, y, m_coeff_x, m_coeff_y);
                if (!m_mesh.is_inner(i, j)) {
                    m_solution[m_mesh.full_index(i, j)] = exact::boundary(x, y);
                }
            }
        }
    }

private:
    const UniformMesh& m_mesh;
    DataType m_coeff_x, m_coeff_y;
    std::vector<DataType> m_right_hand_side;
    std::vector<DataType> m_solution;
};
```

## 7.28 shifted_power_method.h File Reference

```
#include "power_method.h"
#include "laplacian_operator.h"
```

### Classes

- class ShiftedPowerMethod

  *Computes the smallest eigenvalue using the shifted power method.*

## 7.29 shifted_power_method.h

Go to the documentation of this file.

```
#pragma once
#include "power_method.h"
#include "laplacian_operator.h"

class ShiftedPowerMethod {
public:
    template<typename Operator>
    using Params = PowerMethod::Params<Operator>;

    ShiftedPowerMethod(DataType _shift) : m_shift(_shift) { }

    template<typename Operator>
    std::tuple<DataType, unsigned> compute_min(Params<Operator>& params) const {
        PowerMethod power_method;

        class ShiftedOperator {
        public:
            ShiftedOperator(const LaplacianOperator& op, DataType shift)
                : m_op(op), m_shift(shift) {
            }

            void apply(const std::vector<DataType>& x, std::vector<DataType>& y) const {
                m_op.apply(x, y);
#ifdef _OPENMP
#pragma omp parallel for
#endif
                for (std::ptrdiff_t i = 0; i < static_cast<std::ptrdiff_t>(x.size()); ++i) {
                    y[i] = m_shift * x[i] - y[i];
                }
            }

        private:
            const LaplacianOperator& m_op;
            DataType m_shift;
        };
```

```

        ShiftedOperator shifted_op(params.op, m_shift);

        Params shifted_params(shifted_op, params.mesh, params.x, params.tolerance, params.max_iter);

        auto [mu, iterations] = power_method.compute_max(shifted_params);
        return { m_shift - mu, iterations };
    }

private:
    const DataType m_shift;
};
```

## 7.30 solver_controller.h File Reference

```
#include "functions.h"
#include "seidel_solver.h"
#include "power_method.h"
#include "shifted_power_method.h"
#include "initial_vector_type.h"
#include "clock_wrapper.h"
#include "eigen_theory.h"
#include "output_handler.h"
#include "context.h"
#include <random>
```

### Classes

- class SolverController

  *Controller managing the solution of the equation, eigenvalue computation, and output/saving of results.*

## 7.31 solver_controller.h

Go to the documentation of this file.

```
#pragma once
#include "functions.h"
#include "seidel_solver.h"
#include "power_method.h"
#include "shifted_power_method.h"
#include "initial_vector_type.h"
#include "clock_wrapper.h"
#include "eigen_theory.h"
#include "output_handler.h"
#include "context.h"
#include <random>

class SolverController {
public:
    SolverController(double a, double b, double h, double tol_power, double tol_seidel,
        int max_iter_power, int max_iter_seidel)
        : ctx{ UniformMesh(h), a, b, h, tol_power, tol_seidel,
           static_cast<size_t>(max_iter_power), static_cast<size_t>(max_iter_seidel) } {
    }

    void run_all(InitialVectorType type_max = InitialVectorType::UnitConstant,
        InitialVectorType type_min = InitialVectorType::UnitConstant) {
        OutputHandler::print_header(ctx);
        auto solution = solve_seidel();
        OutputHandler::save_full_solution(ctx, "solution.bin", solution);
        OutputHandler::plot_full_solution();
        compute_eigenvalues(type_max, type_min);
    }

    void run_seidel() {
```

```
        OutputHandler::print_header(ctx);
        auto solution = solve_seidel();
        OutputHandler::save_full_solution(ctx, "solution.bin", solution);
        OutputHandler::plot_full_solution();
    }

    void run_eigen(InitialVectorType type_max = InitialVectorType::UnitConstant,
        InitialVectorType type_min = InitialVectorType::UnitConstant) {
        OutputHandler::print_header(ctx);
        compute_eigenvalues(type_max, type_min);
    }

private:
    std::vector<double> solve_seidel() {
        std::cout « "\n--- Seidel method ---\n";
        SeidelSolver seidel(ctx.mesh, ctx.a, ctx.b);
        Clock clock;
        auto solution = std::move(
            seidel.solve(ctx.tol_seidel, ctx.max_iter_seidel)
        );
        std::cout « "Time: " « clock.elapsed().count() « " s\n";
        return solution;
    }

    std::vector<double> generate_initial_vector(InitialVectorType type) const {
        const std::size_t n = ctx.mesh.nodes_per_side();
        const double h = ctx.mesh.step();
        std::vector<double> v(ctx.mesh.total_nodes(), 0.0);

        switch (type) {
        case InitialVectorType::Random: {
            std::mt19937 rng(std::random_device{}());
            std::uniform_real_distribution<double> dist(-1.0, 1.0);
            ctx.mesh.for_each_inner(
                [&](std::size_t, std::size_t, std::size_t idx) {
                    v[idx] = dist(rng);
                }
            );
            break;
        }
        case InitialVectorType::UnitConstant: {
            ctx.mesh.for_each_inner(
                [&](std::size_t, std::size_t, std::size_t idx) {
                    v[idx] = 1.0;
                }
            );
            break;
        }
        case InitialVectorType::Alternating: {
            for (std::size_t j = 1; j < n - 1; ++j) {
                for (std::size_t i = 1; i < n - 1; ++i) {
                    double val = ((i + j) % 2 == 0) ? 1.0 : -1.0;
                    v[ctx.mesh.full_index(i, j)] = val;
                }
            }
            break;
        }
        }
        return v;
    }

    void compute_eigenvalues(InitialVectorType type_max, InitialVectorType type_min) {
        std::cout « "\n--- Eigenvalue computation ---\n";
        LaplacianOperator laplacian(ctx.mesh, ctx.a, ctx.b);

        std::vector<double> initial_max = generate_initial_vector(type_max);
        std::vector<double> initial_min = generate_initial_vector(type_min);

        Clock clock;

        PowerMethod::Params params(laplacian, ctx.mesh, initial_max,
            ctx.tol_power, ctx.max_iter_power);

        PowerMethod power_method;
        auto [lambda_max, iter_max] = power_method.compute_max(params);
        Clock::Elapsed elapsed_max = clock.restart();

        ShiftedPowerMethod shifted_method(lambda_max);
        params.x = initial_min;
        auto [lambda_min, iter_min] = shifted_method.compute_min(params);
        Clock::Elapsed elapsed_min = clock.elapsed();

        double h = ctx.mesh.step();
        auto [max_theor, min_theor] = eigen_theory::lambda_max_min(ctx.a, ctx.b, h);

        std::cout « "\n--- Max eigenvalue ---\n";
        std::cout « "Computed: " « lambda_max « "\n";
```

```
        std::cout « "Theoretical: " « max_theor « "\n";
        std::cout « "Difference: " « std::abs(lambda_max - max_theor) « "\n";
        std::cout « "Iterations: " « iter_max « "\n";
        std::cout « "Time: " « elapsed_max.count() « " s\n";

        std::cout « "\n--- Min eigenvalue ---\n";
        std::cout « "Computed: " « lambda_min « "\n";
        std::cout « "Theoretical: " « min_theor « "\n";
        std::cout « "Difference: " « std::abs(lambda_min - min_theor) « "\n";
        std::cout « "Iterations: " « iter_min « "\n";
        std::cout « "Time: " « elapsed_min.count() « " s\n";
    }

private:
    Context ctx;
};
```